

软考知识点速记

一、专题知识点

- 考点分布

计算机与软件工程知识考试	
科目	重要知识点
软件工程基础知识	开发模型、McCabe复杂度计算、软件维护类型、Gant图与Pert图
面向对象	面向对象基本概念、UML、设计模式
数据结构与算法	栈与队列、树与二叉树、查找与排序、时间复杂度
程序设计语言	正规式、传值与传址调用、有限自动机
计算机组成与体系结构	数据的表示、分层存储体系、流水线
操作系统	前趋图与PV操作、死锁问题、段页式存储与页面置换算法
数据库系统	E-R模型、关系代数、规范化理论（键、范式、模式分解）
计算机网络	TCP/IP协议族、子网划分
信息安全知识	加密技术与数字证书、网络安全协议、防火墙
多媒体基础	媒体的类型
知识产权与标准化	侵权判定，知识产权归属

1、计算机网络

1-1、IP地址划分

类别	点分十进制	二进制	
A类	0.0.0.0	最低	00000000 00000000 00000000 00000000
	127.255.255.255	最高	01111111 11111111 11111111 11111111
B类	128.0.0.0	最低	10000000 00000000 00000000 00000000
	191.255.255.255	最高	10111111 11111111 11111111 11111111
C类	192.0.0.0	最低	11000000 00000000 00000000 00000000
	223.255.255.255	最高	11011111 11111111 11111111 11111111
D类组播	224.0.0.0	最低	11100000 00000000 00000000 00000000
	239.255.255.255	最高	11101111 11111111 11111111 11111111
E类保留	240.0.0.0	最低	11110000 00000000 00000000 00000000
	255.255.255.255	最高	11111111 11111111 11111111 11111111

1-2、网络协议与安全

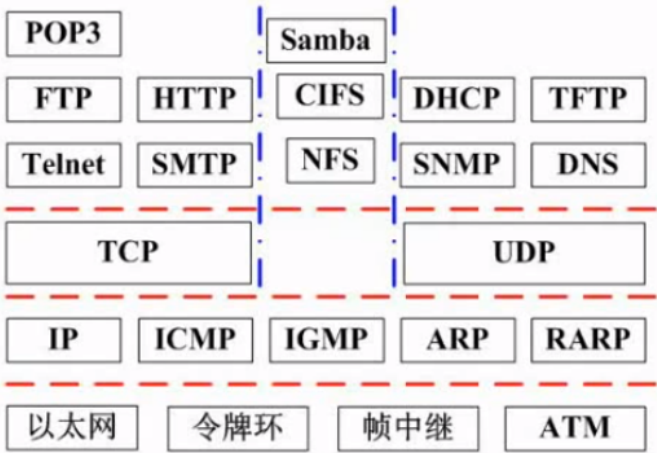
应用层	HTTP、SMTP、SNMP、FTP、Telnet、SIP、SSH、NFS、RTSP、XMPP、Whois、ENRP、等等
表示层	XDR、ASN.1、SMB、AFP、NCP、等等
会话层	ASAP、SSH、RPC、NetBIOS、ASP、Winsock、BSD Sockets、等等
传输层	TCP、UDP、TLS、RTP、SCTP、SPX、ATP、IL、等等
网络层	IP、ICMP、IGMP、IPX、BGP、OSPF、RIP、IGRP、EIGRP、ARP、RARP、X.25、等等
数据链路层	以太网、令牌环、HDLC、帧中继、ISDN、ATM、IEEE 802.11、FDDI、PPP、等等
物理层	例如铜缆、网线、光缆、无线电等等

网络技术标准与协议

- ✓ TCP/IP协议：Internet，可扩展，可靠，应用最广，牺牲速度和效率

✓ IPX/SPX协议：NOVELL，路由，大型企业网

✓ NETBEUI 协议：IBM，非路由，快速



应用层	应用层
	表示层
	会话层
传输层	传输层
Internet层	网络层
网络接口层	数据链路层
	物理层

网络安全 – 各个网络层次的安全保障



1-3、OSI七层模型

物理层 >> 数据链路层 >> 传输层 >> 网络层 >> 会话层 >> 表示层 >> 应用层

1-4、常用协议默认端口

公用端口：0-1024

注册端口：1025-49151

动态或私有端口：49152-65535

协议	默认端口
HTTP	80
HTTPS	443
FTP	21
SFTP	22
SSH	22
Telnet	23
SMTP	25
POP3	110
IMAP	143
DNS	53
DHCP	67/68
SNMP	161/162
NTP	123
LDAP	389
RDP	3389
VNC	5900/5901/5902
MySQL	3306
PostgreSQL	5432
Redis	6379
MongoDB	27017

1-5、常见网络设备

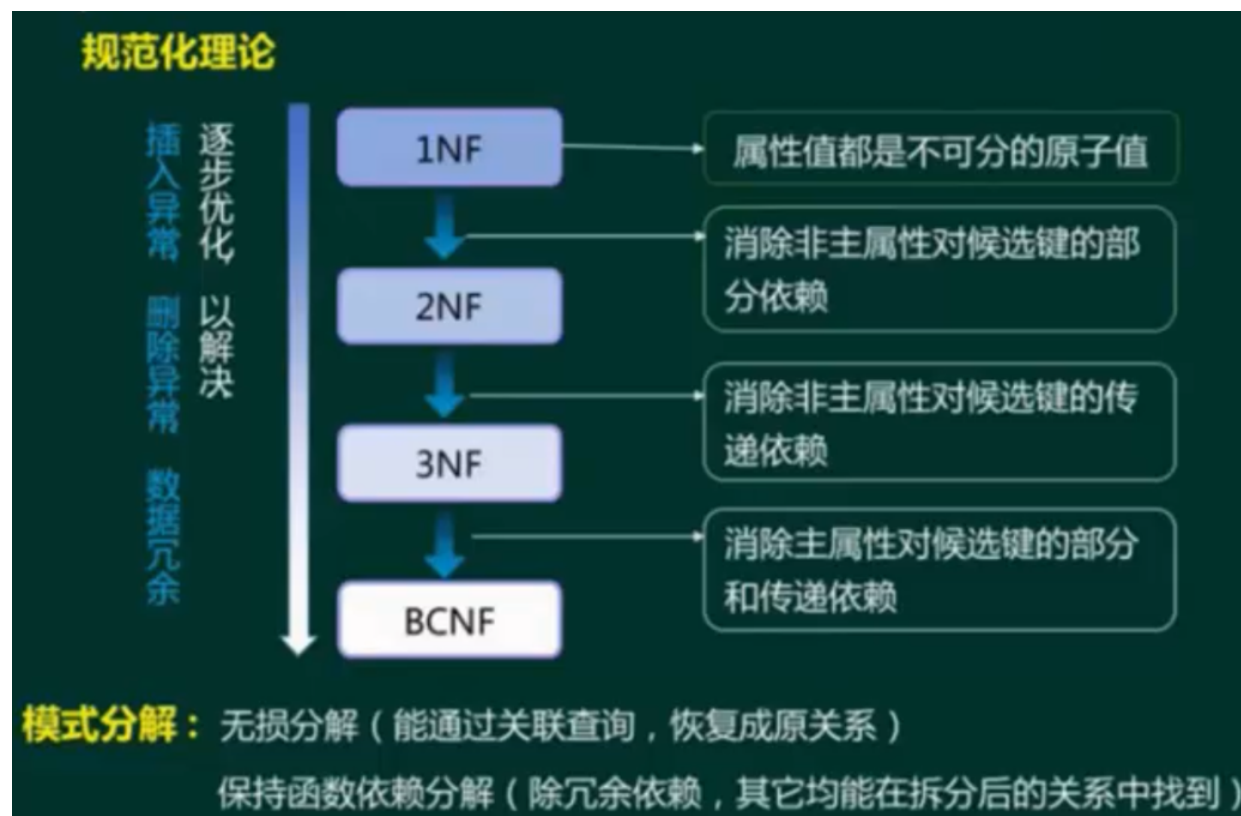
其中注意的是，若非特指某一层的交换机，即可为网络层，也可为数据链路层，甚至是传输层。

设备类型	对应OSI层级
集线器 (Hub)	OSI参考模型第1层 (物理层)
网桥 (Bridge)	OSI参考模型第2层 (数据链路层)
交换机 (Switch)	OSI参考模型第2层 (数据链路层)
路由器 (Router)	OSI参考模型第3层 (网络层)
网关 (Gateway)	OSI参考模型第3层及以上，通常应用在第7层 (应用层)
防火墙 (Firewall)	OSI参考模型第3层及以上，通常应用在第7层 (应用层)
网关交换机 (Gateway Switch)	OSI参考模型第2层 (数据链路层) 和第3层 (网络层)
四层交换机 (Layer 4 Switch)	OSI参考模型第4层 (传输层)
负载均衡器 (Load Balancer)	OSI参考模型第4层 (传输层) 和第7层 (应用层)
DNS服务器 (DNS Server)	OSI参考模型第5层 (会话层) 和第7层 (应用层)
DHCP服务器 (DHCP Server)	OSI参考模型第5层 (会话层) 和第7层 (应用层)
代理服务器 (Proxy Server)	OSI参考模型第5层 (会话层) 和第7层 (应用层)
中继器 (Repeater)	OSI参考模型第1层 (物理层)

2、算法策略

3、数据库系统

3-1、规范化理论



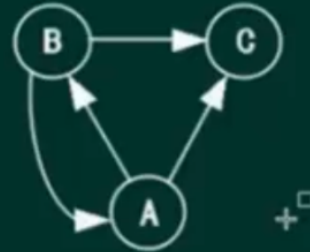
3-2、候选键的求解

选取入度为0的节点，即为候选键。示例中，候选键为B。

规范化理论 – 图示法 求候选键

- 1、将关系的函数依赖关系，用“有向图”的方式表示。
- 2、找出入度为0的属性，并以该属性集合为起点，尝试遍历有向图，若能正常遍历图中所有结点，则该属性集即为关系模式的候选键。
- 3、若入度为0的属性集不能遍历图中所有结点，则需要尝试性的将一些中间结点（既有入度，也有出度的结点）并入入度为0的属性集中，直至该集合能遍历所有结点，集合为候选键。

关系R (A , B , C) 满足下列函数依赖：
F { B → C , B → A , A → BC }



3-3、三级模式-两级映射

三级模式为：外模式（用户模式）、模式（逻辑模式）、内模式（物理模式）

两级映射：外模式-模式的映射、模式-内模式的映射。

外模式可以有任意个，模式和内模式只能有1个

4、操作系统

4-1、PV操作与前趋图

P操作，-1；V操作，+1。

前趋图与PV操作

前趋图



PV操作



单缓冲区生产者、消费者问题PV原语描述：

生产者：

生产一个产品;

P(s1);

送产品到缓冲区;

V(s2);

消费者：

P(s2);

从缓冲区取产品;

V(s1);

消费产品;

+

S1初值为1，S2初值为0

5、程序语言基础

5-1、函数调用

函数调用-传值与传址



传递方式	主要特点
传值调用	形参取的是实参的值，形参的改变不会导致调用点所传的实参的值发生改变
引用（传址）调用	形参取的是实参的地址，即相当于实参存储单元的地址引用因此其值的改变同时就改变了实参的值

6、编译过程

解释型：边解释，边执行。

编译型：源代码->中间代码->目标程序

编译过程

- ✓ 词法错误：非法字符，关键字或标识符拼写错误
- ✓ 语法错误：语法结构出错，if endif不匹配，缺分号
- ✓ 语义错误：死循环，零除数，其它逻辑错误

✓ 解释型

✓ 编译型



7、数据结构

7-1、排序

排序算法	平均时间复杂度	最好情况	最坏情况	空间复杂度	排序方式	稳定性
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	In-place	不稳定
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
希尔排序	$O(n \log n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(1)$	In-place	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Out-place	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	In-place	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	In-place	不稳定
计数排序	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Out-place	稳定
桶排序	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	Out-place	稳定
基数排序	$O(n \times k)$	$O(n \times k)$	$O(n \times k)$	$O(n + k)$	Out-place	稳定

排序

1. 排序的概念

21 32 13 45 27 38 76 21

▼ 稳定与不稳定排序

▼ 内排序与外排序

2. 排序方法分类

▼ 插入类排序

➢ 直接插入排序

➢ 希尔排序

▼ 交换类排序

➢ 冒泡排序

➢ 快速排序

▼ 选择类排序

➢ 简单选择排序

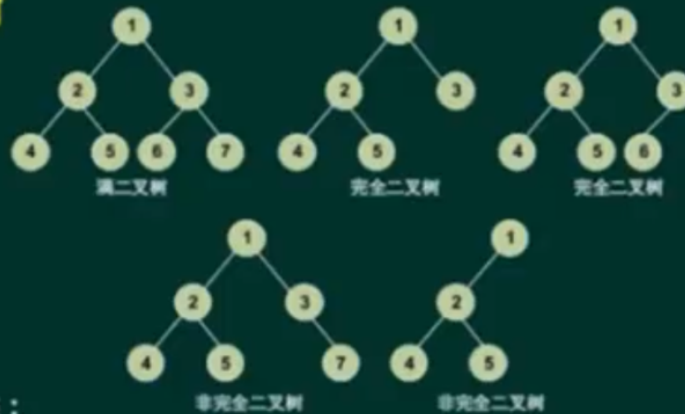
➢ 堆排序

▼ 归并排序

▼ 基数排序

7-2、树与二叉树

树与二叉树



二叉树遍历
二叉排序树
平衡二叉树
最优二叉树

二叉树的重要特性：

- 1、在二叉树的第 i 层上最多有 2^{i-1} 个结点 ($i \geq 1$) ;
- 2、深度为 k 的二叉树最多有 $2^k - 1$ 个结点 ($k \geq 1$) ;
- 3、对任何一棵二叉树, 如果其叶子结点数为 n_0 , 度为2的结点数为 n_2 , 则 $n_0 = n_2 + 1$ 。
- 4、如果对一棵有 n 个结点的完全二叉树的结点按层序编号 (从第1层到 $\lfloor \log_2 n \rfloor + 1$ 层, 每层从左到右), 则对任一结点 i ($1 \leq i \leq n$), 有:
 - ✓ 如果 $i=1$, 则结点 i 无父结点, 是二叉树的根; 如果 $i>1$, 则父结点是 $\lfloor i/2 \rfloor$;
 - ✓ 如果 $2i > n$, 则结点 i 为叶子结点, 无左子结点; 否则, 其左子结点是结点 $2i$;
 - ✓ 如果 $2i+1 > n$, 则结点 i 无右子叶点, 否则, 其右子结点是结点 $2i+1$ 。

8、设计模式

面向对象设计 – 设计模式的分类

➤ 创建型模式

- 工厂方法 (factory method) 模式
- 抽象工厂 (abstract factory) 模式
- 原型 (prototype) 模式
- 单例 (singleton) 模式
- 构建器 (builder) 模式

➤ 结构型模式

- 适配器 (adapter) 模式
- 桥接 (bridge) 模式
- 组合 (composite) 模式
- 装饰 (decorator) 模式
- 外观 (facade) 模式
- 享元 (flyweight) 模式
- 代理 (proxy) 模式

➤ 行为型模式

- 职责链 (chain of responsibility) 模式
- 命令 (command) 模式
- 解释器 (interpreter) 模式
- 迭代器 (iterator) 模式
- 中介者 (mediator) 模式
- 备忘录 (memento) 模式
- 观察者 (observer) 模式
- 状态 (state) 模式
- 策略 (strategy) 模式
- 模板方法 (template method) 模式
- 访问者 (visitor) 模式

9、面向对象技术

9-1、面向对象基本概念

面向对象的基本概念

- ★ 对象：属性 (数据) + 方法 (操作) + 对象ID
- ★ 类 (实体类/控制类/边界类)
- ★ 继承与泛化：复用机制
- ★ 封装：隐藏对象的属性和实现细节,仅对外公开接口
- ★ 多态：不同对象收到同样的消息产生不同的结果
- ★ 接口：一种特殊的类，他只有方法定义没有实现
- ★ 重载：一个类可以有多个同名而参数类型不同的方法

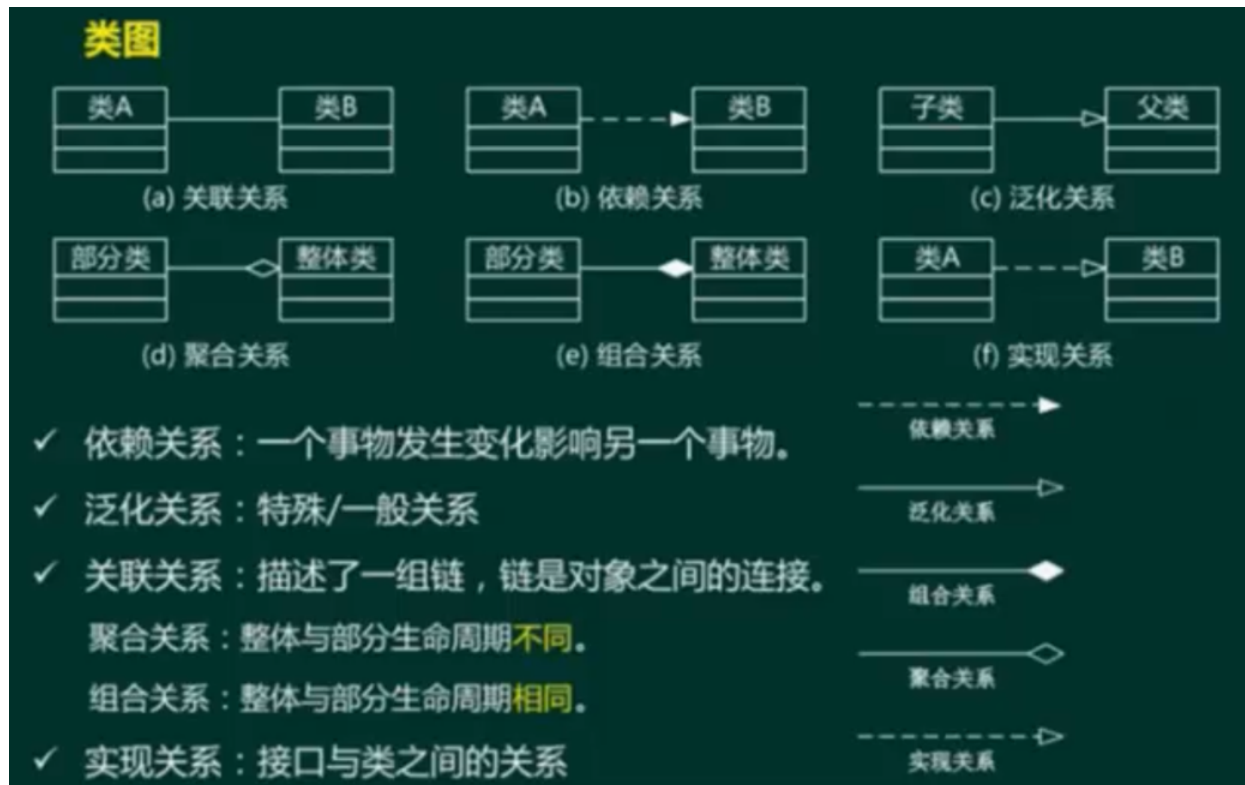
9-2、UML关系类型

UML中存在五种关系：

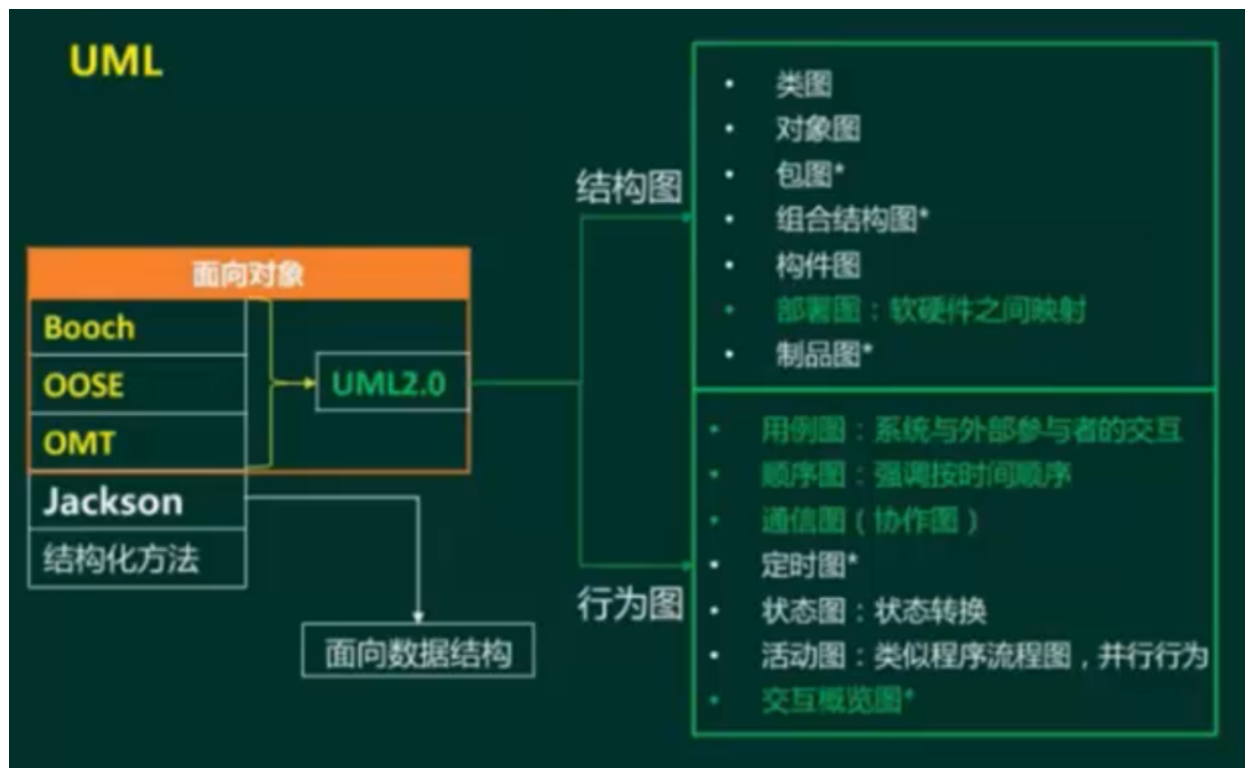
- 关联：
- 依赖
- 泛化

- 聚合
- 组合

以下关系在UML中均适用，不局限于类图。



9-3、UML图



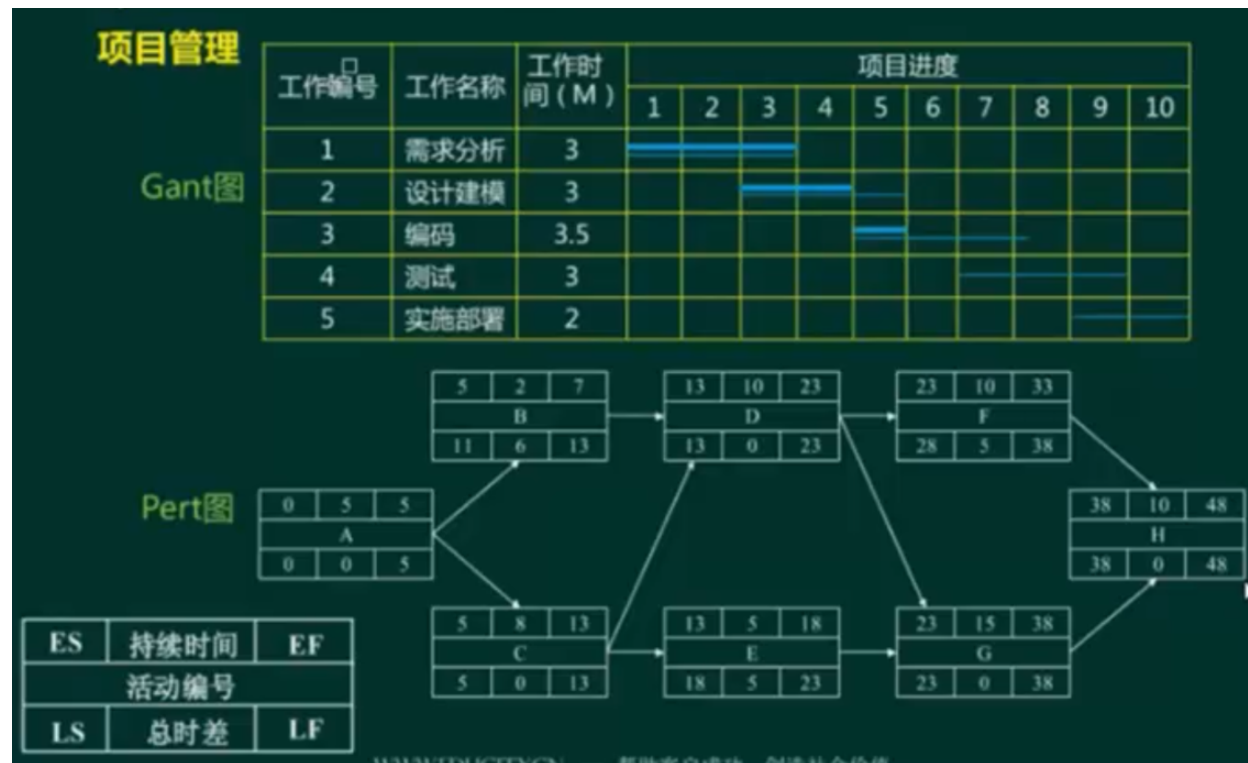
9-4、面向对象设计原则

面向对象设计7大原则

- ★ 单一职责原则：设计目的单一的类
- ★ 开放-封闭原则：对扩展开放，对修改封闭
- ★ 李氏（Liskov）替换原则：子类可以替换父类
- ★ 依赖倒置原则：要依赖于抽象，而不是具体实现；针对接口编程，不要针对实现编程
- ★ 接口隔离原则：使用多个专门的接口比使用单一的总接口要好
- ★ 组合重用原则：要尽量使用组合，而不是继承关系达到重用目的
- ★ 迪米特（Demeter）原则（最少知识法则）：一个对象应当对其他对象有尽可能少的了解

10、项目管理

PERT图与GANT图：

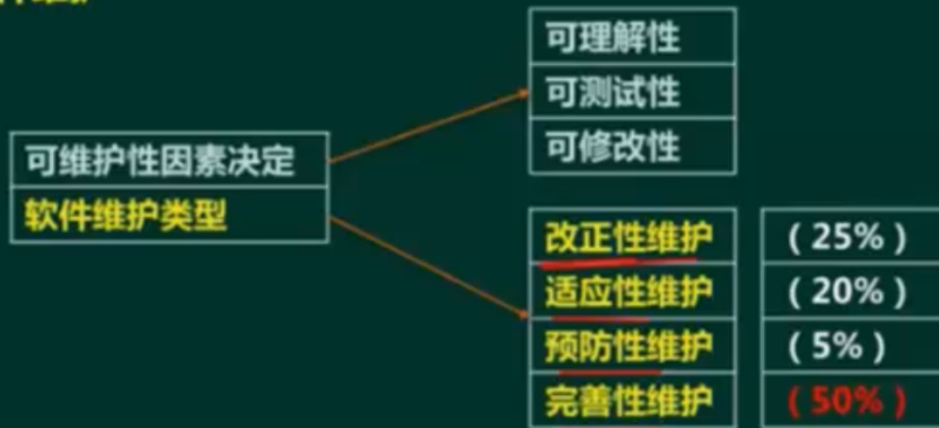


11、系统开发基础

11-1、维护类型

常考点为：适应性维护和改善型维护。

软件维护



★ **适应性维护**。指使应用软件适应信息技术变化和管理需求变化而进行的修改。企业的外部市场环境和管理需求的不断变化也使得各级管理人员不断提出新的信息需求。

★ **完善性维护**。扩充功能和改善性能而进行的修改。对已有的软件系统增加一些在系统分析和设计阶段中没有规定的功能与性能特征。

12、软件工程

12-1、耦合和内聚

1) 耦合类型

- **内容耦合**：当一个模块直接修改或操作另一个模块的数据时，或一个模块不通过正常入口而转入另一个模块时。
- **公共耦合**：两个或两个以上的模块共同引用一个全局数据项。
- **外部耦合**：一组模块都访问同一全局简单变量而不是统一全局数据结构，而且不是通过参数表传递全局变量的信息。
- **控制耦合**：一个模块通过接口向另一个模块传递一个控制信号，接收信号的模块根据信号值进行适当的动作。
- **标记耦合**：一个模块A通过接口向另外两个模块B\C传递一个公共参数，则B和C之间存在标记耦合。
- **数据耦合**：模块之间通过参数来传递数据，即一个模块的输出作为另一个模块的输入。
- **非直接耦合**：两个模块没有直接关系，它们之间联系仅通过主模块调用和控制实现。

2) 内聚类型

- **偶然内聚**：一个模块各成分之间毫无关系，则其为偶然内聚。
- **逻辑内聚**：几个**逻辑上相关**的功能被放在同一模块中，称之为逻辑内聚。
- **时间内聚**：如果一个模块完成的功能必须在同一时间内执行，如系统初始化，但这些功能只是因为时间因素联系在一起。
- **通信内聚**：一个模块的所有成分都操作同一个数据集或生成同一数据集。
- **功能内聚**：模块中的所有成分对于单一的功能都是必须的。
- **信息内聚**：模块完成多个功能，各个功能都在同一数据结构上操作，每一项功能有一个唯一的入口点。

13、多媒体技术

媒体的类型：

- 1、感觉媒体：如光、声音等和感官直接相关的内容
- 2、表示媒体：如各类音视频编码，等用作媒体的表示作用
- 3、表现媒体：如键盘、显示器、鼠标等输入输出媒体内容的设备
- 4、存储媒体：如硬盘、USB等存储设备

5、传输媒体：如光纤、电缆等传输介质

媒体的种类

- ★ **感觉媒体**：指直接作用于人的感觉器官，使人产生直接感觉的媒体。如：声音、图形、图像、动画等。
- ★ **表示媒体**：指为了加工、处理和传输感觉媒体而人为研究、构造出来的一种媒体，常见的有**各种编码方式**，如文本编码、图像编码和声音编码等。
- ★ **显示媒体（表现媒体）**：表现和获取信息的物理设备。如：输入显示媒体键盘、鼠标和麦克风等；输出显示媒体显示器、打印机和音箱等。**输入输出设备**
- ★ **存储媒体**：存储数据的物理设备，如磁盘、光盘和内存等。
- ★ **传输媒体**：传输数据的物理载体，如电缆、光缆和交换设备等。

二、小题知识点

- 1、PC（程序计数器）：存储下一条待执行指令的地址。
- IR（指令寄存器）：存储即将执行的指令。
- MAR（主存地址寄存器）：保存当前CPU所访问的内存单元的地址。
- PSW（状态条件寄存器）：用来存储**状态标志与控制标志**。

- 2、RISC：精简指令集系统计算机
- CISC：复杂指令集系统计算机

指令系统类型	指令	寻址方式	实现方式	其它
CISC（复杂）	数量多，使用频率差别大，可 变长格式	支持多种	微程序控制 技术（微码）	研制周期长
RISC（精简）	数量少，使用频率接近，定长格式，大部分为单周期指令，操作寄存器，只有Load/Store操作内存	支持方式少	增加了通用寄存器；硬布线逻辑控制为主；适合采用流水线	优化编译，有效支持高级语言

- 3、DMA方式直接读取内存，传送数据占用存储周期，与CPU无关。
- 4、闪存(FlashMemory)非采用随机访问方式，无法代替主存也无法代替RAM。
- 5、DMA传送与CPU无关，发送结束信号为处理器外部请求事件，为“中断”；
- 在运行指令时，发生的中断事件，包括错误情况引起的故障，为“异常”。如除零算数错误，缺页异常等
- 6、对称加密算法：DES、3DES、AEC、IDEA、SEED算法、RC4算法、等
- 非对称加密算法：RSA、DSA、ECC、DH算法、ECDH算法、EIGAMAL等
- 摘要算法：SHA、MD5、SHA-1、SHA-256等
- 7、IPSec协议：工作于网络层，为IP数据报文进行加密

PP2P协议：工作于数据链路层，用于链路加密

HTTPS协议：HTTP+SSL协议的结合体，工作于传输层

TLS协议：安全传输层协议，用于两个通信应用程序之间，提供保密性和数据完整性

8、词法分析：处理**非法字符**、**单词拼写错误**等

语法分析：处理**标点符号错误**、**表达式中缺少操作数**、**括号不匹配**等问题

静态语义分析：处理**运算符不合法**、**运算对象类型不合法**等错误

动态语义分析（目标代码执行阶段）：处理动态语义错误，如**陷入死循环**、**变量取零做除数**、**引用数组元素下标越界**等问题

9、ipv6协议，有128位，地址空间为 2^{128} 。示例：FC00:0000:130F:0000:0000:09C0:876A:130B。分为8组，每组4个16进制数。

10、面向对象分析过程：认定对象→组织对象→描述对象间的相互作用→定义对象的操作→定义对象的内部消息

11、MTTF：平均无故障时间、MTTR：平均修复时间

软件可维护性：
$$M = \frac{1}{1+MTTR}$$

软件可靠性：
$$M = \frac{MTTF}{1+MTTF}$$

12、UML关系：6种（依赖：虚线实心三角；泛化：实线空三角；实现：虚线空三角；聚合：实线空心菱心；组合：实线实心菱心；关联：实线三角或无方向实线）

三、大题知识点

– 考点分布

历年考试情况 – 软件设计			
题号	试题类型	学科知识点	考查内容
试题1	必答题	数据流图	补充数据流图的缺失部分（补充数据流、补充外部实体、补充数据存储），数据流图的改错（包括修正数据流名称、数据流的起点与终点、删除多余数据流），与数据流图相关的概念简答题。
试题2	必答题	数据库设计	E-R模型、关系模式、主键、外键、SQL语言。
试题3	必答题	UML建模	用例图、类图与对象图、顺序图、活动图、状态图。
试题4	必答题	C语言算法	链表、栈、二叉树、图基本操作的程序实现、动态规划法、分治法、回溯法、递归法、贪心法
试题5	选答题	C++语言程序设计	C++语法 + 设计模式
试题6	选答题	Java语言程序设计	Java语法 + 设计模式

1、DFD数据流图

结构化语言：一般位于软件设计师下午题第一题第四问。

基本的语句有三类：

- 1、顺序语句
- 2、选择语句

```
1 IF 条件 THEN
2   分支内容
3 ELSE IF 条件 THEN
4   分支内容
5 ELSE
6   分支内容
7 ENDIF
```

3、循环语句

```
1 WHILE 下雨
2 DO
3 {
4   在家
5 IF 不下雨 THEN
6   出门
7 ENDIF
8 }
```

示例：

```
1 # 21年上半年 案例题第一题第四问
2 "道闸控制"加工过程
3 IF (道闸执行状态正常)
4   IF (车辆入场) THEN
5     将车牌号及其入场时间信息存入停车记录，修改空余车位数
6   ELSEIF (车辆出场) THEN
7     更新停车状态，修改空余车位数
8   ENDIF
9 ELSEIF (未在规定的时间内接收到其返回的执行状态正常放行) THEN
10    向管理人员发送异常告警信息
11 ENDIF
```

案例二：

```
1 INPUT 贷款、还贷记录
2 IF 贷款>限额
3   IF 还贷记录良好
4     IF 贷款>2万
5       THEN PRINT 拒绝贷款
6     ELSE PRINT 可作出贷款安排
7   ENDIF
8 ELSE PRINT 拒绝贷款
9 ENDIF
10 ELSE PRINT 允许立即贷款
11 ENDIF
```

注意事项：

1. **命名**：数据流图中**每个元素必须有名字**，且命名合理。

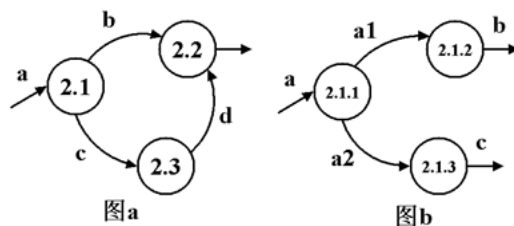
加工名字应该反映整个加工的功能，而不是一部分功能。加工命名一般为**动词+名词短语**。

数据流名字应该代表整个数据流的内容，而不是仅仅反映它的某些成分。数据流命名为**名词**。

2. 画数据流不是画控制流。数据流图反映的是系统“做什么”，不反映“如何做”。

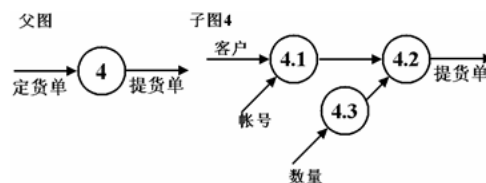
3. **每个加工至少有一个输入数据流和一个输出数据流**，反映出此加工数据的来源与加工的结果。

4. **按层给加工编号**。如果一张数据流图中的某个加工要分解成另一张数据流图时，则上层图为父图，直接下层图为子图，**父、子图上的所有处理都应编号**。子图的编号是父图中相应处理的编号的扩充，子图上处理的编号是父图号、小数点及子图的局部号组成。



5. 保持父图与子图的平衡。子图的输入、输出数据流同父图相应加工的输入、输出数据流必须一致。

注意：如果父图的一个输入（或输出）数据流对应于子图中几个输入（或输出）数据流，而子图中组成这些数据流的数据项全体恰好是父图中的这个数据流，那么父图和子图仍然是平衡的。可借助数据字典中的关于该数据流的描述来判定。



答题技巧：

答题技巧

一、补充实体

实体可能是：

- (1) 人物角色：如 客户、管理员、主管、经理、老师、学生
- (2) 组织机构：如 银行、供应商、募捐机构
- (3) 外部系统：如 银行系统、工资系统、后台数据库（当要开发的是中间件时）

二、补充存储

存储的文字方面特征：“**文件” “**表” “**库” “**清单” “**档案”

三、补充数据流

1、数据平衡原则

- (1) 顶层图与0层图对比，是否有顶层图有，但0层图无的数据流，或反之。
- (2) 检查图中每个加工，是否存在只有入没有出，或只有出没有入，或根据输入的数据无法产生对应的输出的情况。

2、按题目说明与图进行匹配

说明中的每一句话，都能与图中有对应关系，当把说明中的实体与数据流标识出来之后，容易缩小对应范围，找出纰漏。

四、补充加工名

加工是用于处理数据流的，所以要补充加工名，可以把该加工涉及到的数据流，在说明中标识出来，再在数据流名称所在的句子中，找“动词+名词”的结构，分析是否可作为加工。

“动词+名词”如：生成报告，发出通知，批改作业，记录分数，当然这只是普遍情况，也有例外，如物流跟踪、用户管理

2、数据库设计

1、完整性约束关系指的是：找出关系中的主键以及外键。

注意事项：

- 1、此类问题需注意前后关系属性名需保持一致。
- 2、找寻一个关系的主键，查找关系中入度为0的属性。
- 3、外键：能够在其他关系中作为主属性存在的一个外码。

3、UML图

注意事项：

- 1、部分实体可能不会在说明中体现，需结合实际生活场景补充实体名称。

4、C语言及算法设计

题目类型基本为：程序填空-8分、算法类型、算法复杂度，算法的计算结果。重点关注非程序填空题-7分。

常见算法特征总结

➤ 一、分治法

特征：把一个问题拆分成多个小规模的不同子问题，一般可用递归解决。

经典问题：

斐波那契数列、归并排序、快速排序、矩阵乘法、二分搜索、大整数乘法、汉诺塔

➤ 二、动态规划法（用于求最优解）

特征：划分子问题，并把子问题结果使用数组存储，利用查询子问题结果构造最终问题结果。

经典问题：

斐波那契数列、矩阵乘法、背包问题、LCS最长公共子序列

三、回溯法

特征：系统的搜索一个问题的所有解或任一解。

经典问题：

+ N皇后问题、迷宫、背包问题

四、贪心法（用于求满意解）

特征：局部最优，但整体不见得最优。每步有明确的，既定的策略。

经典问题：

背包问题（如装箱）、多机调度、找零钱问题

时间复杂度排序：

时间复杂度总结

$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n)$

一、常数级时间复杂度 $O(1)$

1、单个语句

如： $k = 0$ ；

2、整个程序都没有循环语句，或复杂函数的调用

```
void main() {  
    char*x= "ABCADAB";  
    int m = strlen(x);  
    printf("len:%d\n",m);  
}
```


二、时间复杂度 $O(n)$

单层循环

```
void main() {  
    int i, j;  
    k = 0;  
    for( i = 0; i < n; i++){  
        b[i] = 0;  
    }  
}
```

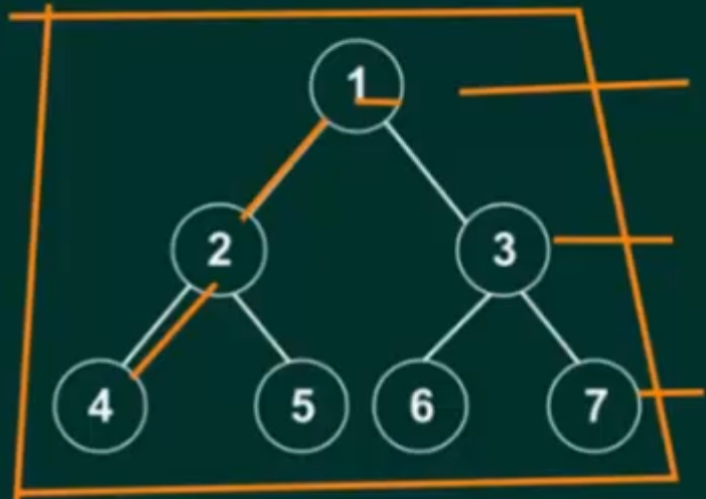
三、时间复杂度 $O(n^2)$

双层循环

```
void main() {  
    int i, s=0, n=1000;  
    for(i=1; i < n; i++)  
        for(j=1; j < n; j++)  
            s+=j;  
    printf("结果为:%d",s)  
}
```

四、时间复杂度 $O(\log_2 n)$

```
int search(int array[], int n, int v)  
{  
    int left, right, middle;  
    left = 0, right = n - 1;  
    while (left <= right)  
    {  
        middle = (left + right) / 2;  
        if (array[middle] > v)  
        {  
            right = middle;  
        }  
        else if (array[middle] < v)  
        {  
            left = middle;  
        }  
        else  
        {  
            return middle;  
        }  
    }  
    return -1;  
}
```



五、时间复杂度 $O(n \log_2 n)$

典型代表：堆排序，每次重建堆的时间复杂度是 $\log_2 n$, n 个元素基本上就是 $n \log_2 n$ 。

5、C++程序设计：面向对象

6、Java程序设计：面向对象

常考设计模式

面向对象特点：

➤ 一、定义类型问题较多

关键点：

- 1、interface , implements , extends , abstract等关键字。
- 2、接口中定义方法不能写方法体。

如：void Insert(Department department) ✓

void Insert(Department department){} ✗

+

➤ 二、结合设计模式考查

特点：如果懂该设计模式，则事半功倍，能快速解题；

如果不懂模式，则需要注意：考题中除定义类的空，90%填空内容并非真正解决业务逻辑问题，而是通过调用其它模块完成功能。

换句话说，我们是要完成设计模式中类或对象之间的组装。

Cloneable也特指接口，注意：一般以interface关键字为准，接口会首字母标识为I：

➤ 三、代码均不完整

特点：由于设计模式一般涉及多个接口与类，其自身已有一定量的代码，所以一般会省略一些代码，只留部分代码。□

关键点：

分析填空时，需要尝试把调用时的代码写出来，调用时的代码往往是把各个类串起来的位置。如果这些代码缺失，会让整个结构显得十分的零散，从而加大分析难度。

➤ 四、特征判断

当名称中出现“I**” 开头或 “**able” 结尾时，进一步验证是否为接口。

如：

```
interface Ifactory{
```

```
public class workExperience (1) Cloneable{ //工作经历
```